РЕШЕН ИЗПИТ 2 — С ПЪЛНИ ОБЯСНЕНИЯ

Базови Програмни Езици — C (plain C)

Изпит 2: Болница — Медицинско досие | 100 точки

Теми: четене от БИНАРЕН файл с header-count | броене по условие | realloc с NULL при грешка | запис в текстов файл с форматиран изход

Специфика: add_new_treatment() трябва да върне NULL при грешка — не exit()!

СЕКЦИЯ 0 — ПРОЧИТ И АНАЛИЗ

КОД / ИЗРАЗ	КАКВО ОЗНАЧАВА ЗА НАС
`Идентификационен номер (цяло число)`	int id; — обикновен int
`Дата ДД.ММ.ГГГГ (фиксиран размер)`	char date[11]; — 10 символа + \0 = 11
`Име на пациент (до 30 символа)`	char name[31]; — 30 + 1 = 31
`Диагноза (до 50 символа)`	char diagnosis[51]; — 50 + 1 = 51
`файлът съдържа БРОЯ, последван от записите`	Четем ПЪРВО count с fread, ПОСЛЕ масива с fread в цикъл
`Функцията връща NULL при грешка`	return NULL; вместо exit(1) — важна разлика!

СЕКЦИЯ 1 — ПЪЛЕН КОД (gcc -Wall -std=c99)

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    int  id;              /* идентификационен номер */
    char date[11];       /* ДД.ММ.ГГГГ = 10 + '\0' */
    char name[31];       /* до 30 символа + '\0' */
    char diagnosis[51]; /* до 50 символа + '\0' */
} Treatment;

/* — ЗАДАЧА 1: четене от бинарен файл с header-count — */
Treatment* loadHistory(const char *fn, int *n) {
    FILE *f = fopen(fn, "rb");
    if (!f) { *n = 0; return NULL; }

    /* Стъпка 1: четем БРОЯ на записите */
    if (fread(n, sizeof(int), 1, f) != 1) {
        fclose(f); *n = 0; return NULL;
    }

    /* Стъпка 2: заделяме памет за ВСИЧКИ записи */
    Treatment *arr = (Treatment*)malloc(*n * sizeof(Treatment));
    if (!arr) { fclose(f); *n = 0; return NULL; }

    /* Стъпка 3: четем ВСИЧКИ записи наведнъж */
    size_t rd = fread(arr, sizeof(Treatment), *n, f);
    if ((int)rd != *n) {
        printf("Предупреждение: очаквани %d, прочетени %zu\n", *n, rd);
        *n = (int)rd;  /* работим с реално прочетените */
    }
    fclose(f);
    return arr;
}

/* — ЗАДАЧА 2: count_treatment — */
int count_treatment(Treatment *arr, int n,
                    const char *name, const char *diagnosis) {
    int cnt = 0;
    for (int i = 0; i < n; i++) {
        if (strcmp(arr[i].name, name) == 0 &&
            strcmp(arr[i].diagnosis, diagnosis) == 0)
            cnt++;
    }
    return cnt;  /* 0 ако не е намерен */
}

/* — ЗАДАЧА 3: add_new_treatment — */
Treatment* add_new_treatment(Treatment *arr, int *n) {
    Treatment *tmp = (Treatment*)realloc(arr, (*n + 1) * sizeof(Treatment));
    if (!tmp) {
        printf("Грешка при добавяне!\n");
        return NULL;  /* НЕ exit(1) — задачата иска NULL! */
    }
    arr = tmp;
    (*n)++;
    Treatment *t = &arr[*n - 1];
    printf("ID: ");         scanf("%d",   &t->id);
```

```c
    printf("Дата: ");        scanf("%10s",  t->date);
    printf("Пациент: ");     scanf("%30s",  t->name);
    printf("Диагноза: ");    scanf("%50s",  t->diagnosis);
    return arr;
}

/* ── ЗАДАЧА 4: write_text_file ── */
int write_text_file(Treatment *arr, int n, const char *diagnosis) {
    FILE *f = fopen("illness.txt", "w");
    if (!f) { printf("Грешка при illness.txt!\n"); return 0; }
    int written = 0;
    for (int i = 0; i < n; i++) {
        if (strcmp(arr[i].diagnosis, diagnosis) == 0) {
            fprintf(f, "Болничен престой на %s\n", arr[i].name);
            fprintf(f, "за лечение на %s:\n", arr[i].diagnosis);
            fprintf(f, "%sg.\n", arr[i].date);
            written++;
        }
    }
    fclose(f);
    return written;  /* 0 ако няма съвпадение */
}

int main() {
    int n = 0;
    Treatment *arr = loadHistory("history.bin", &n);
    if (!arr) { printf("Грешка!\n"); return 1; }

    int cnt = count_treatment(arr, n, "Петър Иванов", "Дископатия");
    printf("Брой посещения: %d\n", cnt);

    arr = add_new_treatment(arr, &n);
    if (!arr) { printf("Добавянето се провали.\n"); return 1; }

    int saved = write_text_file(arr, n, "Дископатия");
    printf("Записани: %d\n", saved);

    free(arr);
    return 0;
}
```

СЕКЦИЯ 2 — ЧЕТЕНЕ ОТ БИНАРЕН ФАЙЛ С HEADER-COUNT

Изпит 2 има специфичен формат: файлът history.bin съдържа ПЪРВО броя на записите (int), а ПОСЛЕ самите записи. Трябва два отделни fread — един за брояча, един за данните.

```
/* Структура на history.bin:                        */
/* [ int n ] [ Treatment 0 ] [ Treatment 1 ] ...   */
/*   4 байта     ~102 байта       ~102 байта         */
```

КОД	ОБЯСНЕНИЕ
`fread(n, sizeof(int), 1, f)`	Четем int брояча от началото на файла. Указателят на файла се позиционира след 4-те байта.
`malloc(*n * sizeof(Treatment))`	Заделяме точно колкото записи има. Използваме malloc (не realloc) — знаем размера предварително.
`fread(arr, sizeof(Treatment), *n, f)`	Четем ВСИЧКИ записи наведнъж в масива. По-ефективно от цикъл с по 1 запис.
`size_t rd = fread(...)`	Запазваме броя реално прочетени. Ако файлът е повреден и rd != *n, коригираме.

> **ВАЖНО:** malloc(*n * sizeof(Treatment)) е по-добро от realloc(NULL, ...) тук — знаем точно колко памет ни трябва и го заделяме наведнъж.

Задача 3: add_new_treatment() — return NULL вместо exit()

Критична разлика от другите изпити: задачата ИЗРИЧНО казва 'при грешка функцията връща NULL'. Не трябва exit(1)!

```
[NE]  if (!tmp) { fprintf(stderr, "realloc failed\n"); exit(1); }  // НЕ за Изпит 2!
[OK]  if (!tmp) { printf("Грешка!\n"); return NULL; }              // правилно за Изпит 2
```

КОД	ОБЯСНЕНИЕ
`Treatment *tmp = realloc(arr, (*n+1)*sizeof(Treatment));`	Използваме временна променлива tmp — ако realloc се провали, arr остава валиден и може да се ползва.
`if (!tmp) { return NULL; }`	Нулев указател при грешка. Старият arr е все още валиден — не се губи!
`arr = tmp; (*n)++;`	Само СЛЕД успешен realloc обновяваме arr и брояча.

Задача 4: write_text_file() — форматиран изход

Форматът е фиксиран от условието. Всеки съвпадащ запис се принтира в три реда: 'Болничен престой на <name>', 'за лечение на <diagnosis>:', '<date>г.'

КОД	ОБЯСНЕНИЕ
`fopen("illness.txt", "w")`	"w" = write — изтрива предишното съдържание. Правилно за export на всички съвпадения.
`strcmp(arr[i].diagnosis, diagnosis) == 0`	strcmp за сравнение — НИКОГА ==.
`return written;`	Функцията ВРЪЩА брой записани. Ако 0 = няма съвпадения.

Грешки характерни за Изпит 2

```
[NE]  return NULL; // без fclose(f) преди return — FILE не се освобождава!
[OK]  fclose(f); *n = 0; return NULL;  // затвори файла ПРЕДИ return
```

```
[NE]  exit(1);  // add_new_treatment трябва return NULL, не exit!
[OK]  return NULL;  // задачата изрично изисква това
```